

Zastosowanie MCTS/UCT do gry Breakthrough 8×8

Raport końcowy

Hubert Sobociński

Sebastian Rydz

15 czerwca 2026

Projekt bada skuteczność algorytmu Monte Carlo Tree Search (UCT) oraz jego rozszerzeń (RAVE, Progressive Bias) w grze Breakthrough na planszy 8×8 , zestawiając je dodatkowo z deterministyczną heurystyką alpha-beta. Silnik gry i agentów zaimplementowano w Rust z interfejsem do Pythona, co pozwoliło rozegrać ponad 10 000 partii w serii turniejów.

W sześciu eksperymentach sprawdziliśmy m.in. czy rozszerzenia UCT poprawiają siłę gry, jak heurystyka wypada względem MCTS przy różnych budżetach iteracji, czy pierwszy gracz ma przewagę oraz jak na wynik wpływa stała eksploracji c .

Najważniejsze obserwacje: rozszerzenia opracowane dla gry Go nie przenoszą się wprost na Breakthrough – RAVE nie poprawia gry, a Progressive Bias nie daje istotnej różnicy. Najsilniejsza okazała się prosta heurystyka alpha-beta, która jest zarazem szybsza i skuteczniejsza od UCT w całym praktycznym zakresie.

Spis treści

1	Wstęp	3
1.1	Gra Breakthrough	3
1.2	Dlaczego gra Breakthrough 8×8	3
1.3	Cel projektu	3
1.4	Hipotezy badawcze	3
2	Przegląd literatury	4
2.1	Monte Carlo Tree Search i algorytm UCT	4
2.2	Rozszerzenia MCTS istotne dla projektu	5
2.3	Breakthrough w literaturze MCTS	5
3	Metody rozwiązywania	6
3.1	Reprezentacja stanu gry	6
3.2	Bazowy UCT	6
3.3	UCT + RAVE	6
3.4	UCT + Progressive Bias	6
3.5	Heurystyczny gracz alpha-beta	6
3.6	Architektura implementacji	7
3.7	Interfejs graficzny (człowiek vs AI)	7
4	Plan eksperymentów	9
4.1	Wspólna konfiguracja	9
4.2	H1: RAVE vs bazowy UCT	9
4.3	H2: Progressive Bias vs bazowy UCT	10
4.4	H3: Heurystyka vs bazowy UCT	10
4.5	H4: Przewaga pierwszego gracza	10
4.6	H5: Strojenie stałej eksploracji c	10
4.7	H6: Czas decyzji w funkcji numeru ruchu	10
5	Wyniki	11
5.1	H1: RAVE vs bazowy UCT	11
5.2	H2: Progressive Bias vs bazowy UCT	12
5.3	H3: Heurystyka alpha-beta vs UCT	13
5.4	H4: Przewaga pierwszego gracza	14
5.5	H5: Stała eksploracji c	15
5.6	H6: Czas decyzji w funkcji numeru ruchu	16
5.7	Analizy przekrojowe	17
5.8	Partie człowiek vs AI	18
6	Wnioski	20
6.1	Synteza i kluczowe odkrycie	20
6.2	Propozycje dalszych prac	21
7	Wykorzystanie modeli językowych LLM	21

1 Wstęp

1.1 Gra Breakthrough

Breakthrough to dwuosobowa gra strategiczna z pełną informacją, wynaleziona przez Dana Troyka w 2000 roku (zwycięzca konkursu 8×8 Game Design Competition). Plansza jest kwadratowa, taka jak w szachach; każdy gracz dysponuje pionkami ustawionymi w dwóch pierwszych rzędach po swojej stronie. Ruch pionka odbywa się o jedno pole do przodu – prosto lub po skosie. Ruch prosty jest możliwy wyłącznie na puste pole; ruch po skosie może być wykonany na pole puste lub zajęte przez pionek przeciwnika (bicie). Bicie prosto jest niedozwolone, a pionki nie mogą się cofać.

Gra kończy się zwycięstwem gracza, który:

1. doprowadzi pionek do ostatniego rzędu przeciwnika,
2. zbije wszystkie pionki przeciwnika,
3. uniemożliwi przeciwnikowi wykonanie jakiegokolwiek legalnego ruchu.

Remis nie jest możliwy.

1.2 Dlaczego gra Breakthrough 8×8

Gra spełnia wszystkie wymagania projektu:

- **Dwuosobowa, z pełną informacją, deterministyczna** – kluczowe właściwości dla metod MCTS/UCT.
- **Umiarkowany branching factor** – w typowej pozycji na planszy 8×8 liczba legalnych ruchów wynosi 15–30. Głębokość partii to 40–80 ruchów.
- **Proste zasady, ale trudna strategicznie** – mimo prostych zasad wymaga planowania na wiele ruchów naprzód (tworzenie łańcuchów pionków, kontrola kolumn, poświęcenia pionka w celu wygranej).
- **Niszowa** – w odróżnieniu od szachów czy warcabów gra nie jest aż tak popularna.
- **Wariant 8×8 nie został rozwiązany** – Saffidine i in. [1] rozwiązyli warianty do planszy 6×5, a Björnsson i in. [2] rozwiązali planszę 6×6 (w obu przypadkach wygrywa pierwszy gracz). Klasyczna plansza 8×8 pozostaje otwarta, co zostawia miejsce na eksperymenty porównawcze bez znanego „perfekcyjnego gracza”.

1.3 Cel projektu

Celem projektu jest zaprojektowanie i implementacja agentów grających w Breakthrough 8×8 z wykorzystaniem algorytmu Monte Carlo Tree Search w wariantcie UCT oraz jego rozszerzeń, a następnie eksperymentalne porównanie ich skuteczności.

W szczególności projekt obejmuje:

- implementację bazowego algorytmu UCT,
- implementację dwóch rozszerzeń UCT:
 - RAVE (z literatury),
 - Progressive Bias (z literatury),
- implementację gracza heurystycznego (minimax alfa-beta),
- porównanie jakości gry powyższych podejść,
- stworzenie interfejsu człowiek–komputer i przeprowadzenie testów z użytkownikami.

1.4 Hipotezy badawcze

W projekcie weryfikujemy sześć hipotez:

- H1 RAVE poprawia skuteczność względem bazowego UCT.** Agent UCT rozszerzony o RAVE osiąga istotnie wyższy win-rate niż czysty UCT przy tym samym budżecie iteracji.

- H2 Progressive Bias poprawia skuteczność względem bazowego UCT.** Agent UCT z Progressive Bias osiąga istotnie wyższy win-rate niż czysty UCT, szczególnie przy niskich budżetach iteracji.
- H3 Heurystyka specjalistyczna przewyższa MCTS przy małym budżecie iteracji, jednak przegrywa przy wysokim.** Agent minimax alfa-beta z heurystyką domenową wygrywa z UCT przy małych budżetach iteracji, ale przegrywa przy dużych.
- H4 Przewaga pierwszego gracza.** W Breakthrough pierwszy gracz ma niewielką przewagę – podobna sytuacja jak w szachach.
- H5 Stała eksploracji c w UCT ma nieliniowy wpływ na siłę gry.** Istnieje optymalna wartość c^* maksymalizująca win-rate agenta UCT w Breakthrough 8×8 . Zarówno zbyt niska (nadmierna eksploatacja), jak i zbyt wysoka (nadmierna eksploracja) wartość c prowadzi do istotnego spadku skuteczności.
- H6 Czas podejmowania decyzji MCTS rośnie w środkowej fazie gry.** Przy stałym budżecie iteracji średni czas na ruch agenta MCTS jest najwyższy w fazie środkowej partii (ruchy 15–40), gdy branching factor jest największy, i maleje w otwarciu oraz końcówce.

2 Przegląd literatury

2.1 Monte Carlo Tree Search i algorytm UCT

Algorytm Monte Carlo Tree Search (MCTS), spopularyzowany przez Couloma [3] w kontekście gry Go, to asymetryczna metoda przeszukiwania drzewa decyzyjnego, która w wariantach podstawowym nie wymaga wbudowanej, eksperckiej funkcji oceny heurystycznej. Działanie algorytmu opiera się na iteracyjnym powtarzaniu czterech faz:

1. **Selekcja.** Algorytm przechodzi od korzenia w dół drzewa, wybierając węzły na podstawie tzw. polityki drzewa (tree policy), aż do osiągnięcia liścia lub węzła, który nie został w pełni rozwinięty.
2. **Ekspansja.** Do drzewa dodawany jest nowy węzeł potomny, odpowiadający jednemu z możliwych ruchów z wybranego stanu.
3. **Symulacja.** Z nowo dodanego węzła rozgrywana jest losowa partia (z użyciem tzw. default policy) aż do osiągnięcia stanu terminalnego.
4. **Propagacja wsteczna.** Wynik symulacji (np. wygrana, przegrana) jest propagowany w górę drzewa, aktualizując statystyki odwiedzin i zwycięstw każdego węzła na ścieżce.

Kocsis i Szepesvári [4] zaproponowali algorytm UCT (Upper Confidence Bounds applied to Trees), który optymalizuje fazę selekcji, wykorzystując regułę UCB1, aby zachować balans pomiędzy eksploracją (badaniem nowych ruchów) a eksploatacją (wybieraniem ruchów znanych jako dobre). Wartość wyboru węzła i obliczana jest ze wzoru:

$$UCT_i = \frac{w_i}{n_i} + c \sqrt{\frac{\ln N}{n_i}}, \quad (1)$$

gdzie w_i to liczba zwycięstw, n_i to liczba odwiedzin węzła potomnego, N to liczba odwiedzin węzła rodzica, a c to stała eksploracji. Kompleksowy przegląd MCTS i jego wariantów przedstawili Browne et al. [5], stanowiący podstawowy punkt odniesienia metodologicznego dla niniejszego projektu.

Wykorzystanie w projekcie. Zaimplementujemy klasyczny algorytm UCT, który posłuży jako punkt odniesienia (baseline). Pozwoli on sprawdzić, jak czyste przeszukiwanie losowe radzi sobie w Breakthrough, oraz stanowić będzie podstawę do oceny zysków z zaimplementowanych rozszerzeń.

2.2 Rozszerzenia MCTS istotne dla projektu

RAVE i statystyki AMAF. RAVE (Rapid Action Value Estimation), technika wprowadzona przez Gelly ego i Silvera [6], służy do radykalnego przyspieszenia zbieżności algorytmu poprzez uogólnianie i współdzielenie informacji o jakości ruchów, co opiera się na heurystyce AMAF (All-Moves-As-First). Zamiast aktualizować statystyki danego ruchu tylko w konkretnym węźle drzewa, AMAF zakłada, że jeśli ruch był skuteczny w dowolnym momencie rozegranej symulacji (playoutu), to prawdopodobnie jest to ruch wartościowy również z poziomu węzła początkowego. RAVE modyfikuje politykę drzewa, łącząc klasyczne oszacowanie MCTS z oszacowaniem AMAF poprzez średnią ważoną:

$$V = (1 - \beta) \cdot V_{UCT} + \beta \cdot V_{AMAF}. \quad (2)$$

Waga β jest dynamicznie dobierana i zależy od liczby odwiedzin. Przy małej liczbie symulacji β jest bliskie 1 (dominuje AMAF, co daje szybkie, choć zgrubne rozeznanie), a w miarę wzrostu statystyk β dąży do 0 (dominuje dokładna ocena z UCT).

Wykorzystanie w projekcie: W grze Breakthrough ruchy pionków wzdłuż tych samych kolumn i ścieżek często zachowują swoją skuteczność bez względu na dokładną kolejność innych posunięć. Zaimplementowanie RAVE jest próbą wykorzystania tej właściwości do drastycznego poprawienia siły gry agenta przy niskich budżetach iteracyjnych.

Progressive Bias. Progressive Bias to technika z rodziny strategii progresywnych (opisanych m.in. przez Chaslot i in. [7]), służąca do wstrzykiwania z góry zdefiniowanej wiedzy domenowej (heurystyki) bezpośrednio do fazy selekcji w drzewie MCTS. Zamiast rozpoczynać przeszukiwanie „na ślepo”, algorytm na początku faworyzuje te gałęzie, które z perspektywy funkcji oceny wydają się najlepsze. Formułę selekcji modyfikuje się poprzez dodanie elementu uwzględniającego wartość heurystyczną H_i dla danego ruchu i :

$$\text{Wartość} = UCT_i + \frac{H_i}{n_i + 1}. \quad (3)$$

Kluczowym elementem tego równania jest mianownik. Początkowo parametr H_i ma ogromny wpływ na wybór ścieżki. W miarę gromadzenia statystyk i wzrostu wartości n_i (liczby odwiedzin węzła), rola heurystyki płynnie zanika na rzecz symulacji Monte Carlo. Pozwala to na połączenie mocnych stron obu podejść: silnego początku gwarantowanego przez wiedzę ludzką oraz teoretycznej zbieżności do optimum zapewnianej przez MCTS.

Wykorzystanie w projekcie: Zbudujemy specjalistyczną funkcję heurystyczną dla Breakthrough i włączymy ją w algorytm MCTS poprzez Progressive Bias. Pozwoli to zbadać, czy ukierunkowanie MCTS w początkowej fazie daje lepsze rezultaty niż poleganie na wariancie uczącym się od zera.

2.3 Breakthrough w literaturze MCTS

State of the art Breakthrough. Saffidine et al. [1] opracowali techniki „race patterns” i Job-Level Proof Number Search, pozwalające im rozwiązać warianty Breakthrough do planszy 6×5 łącznie; we wszystkich *wygrywa pierwszy gracz*. Niedawno Björnsson et al. [2] rozwiązyli również planszę 6×6 (także na korzyść pierwszego gracza). Klasyczna plansza 8×8 pozostaje natomiast nierozwiązana – nie ma znanego solwera o ograniczonych zasobach pamięciowych zdolnego do pełnego przeszukania jej drzewa. To czyni Breakthrough 8×8 atrakcyjnym benchmarkiem: nie istnieje wyrocznia „perfekcyjnego gracza” definiująca sufit jakości, więc porównanie wariantów MCTS ma realną wartość badawczą.

Heurystyki ewaluacji. Lorentz [8] w swojej pracy opisuje proste heurystyki ewaluacji pozycji zawierające zaawansowanie pionków, materiał oraz bezpieczeństwo struktury (pionki na krawe-

dziach są bezpieczniejsze, ponieważ mogą zostać zbite tylko z jednej strony zamiast dwóch). Te elementy stanowią podstawę naszej implementacji `HeuristicAgent`.

3 Metody rozwiązywania

3.1 Reprezentacja stanu gry

Stan gry Breakthrough w naszej implementacji jest reprezentowany przez parę bitboardów u_{64} : jeden dla pionków białych, jeden dla czarnych. Bit i odpowiada polu $i = r \cdot c + k$, gdzie r to numer rzędu (0-indeksowany od dołu), k to numer kolumny, a c to liczba kolumn planszy. Dla planszy 8×8 wszystkie 64 pola zajmują dokładnie całą szerokość `u64`.

Walidacja krzyżowa. Dla pewności poprawności silnika, równolegle z implementacją w Rustie została napisana referencyjna implementacja w Pythonie. Test `test_rust_vs_python.py` gra 1000 losowych partii i porównuje po każdym ruchu zbiory legalnych ruchów, stan terminalny, zwycięzcę i reprezentację tekstową.

3.2 Bazowy UCT

Bazowy 4-fazowy algorytm MCTS z regułą selekcji UCB1 opisaliśmy w sekcji 2; tutaj odnotowujemy szczegóły implementacyjne. Statystyki $wins(c)$ liczymy z perspektywy gracza, który wykonał ruch c – konwencja ta czyni selekcję UCB1 minimax-spójną. Symulacja jest losowa (uniform) do stanu terminalnego. Pula węzłów to płaska `Vec<UctNode>` indeksowana `usize`; węzły nie przechowują pełnego `GameState`, lecz stan jest rekonstruowany przy schodzeniu od korzenia (koszt $O(\text{depth})$ tanich klonowań na iterację). Eksponowany do Pythona przez PyO3 jako `UctAgent(iterations, c, seed)`.

3.3 UCT + RAVE

Mechanizm RAVE/AMAF i formułę mieszającą oceny UCT z AMAF omówiliśmy w sekcji 2. Implementacyjnie każdy węzeł s przechowuje dodatkowe statystyki AMAF w mapie kluczowanej kodem ruchu (`from*64 + to`). W backpropagation aktualizujemy je dla ruchów zagranych zarówno w fazie symulacji, jak i *in-tree* poniżej węzła – włączenie ruchów *in-tree* okazało się kluczowe; ich pominięcie w pierwszej wersji prowadziło do drastycznej porażki RAVE (patrz sekcja 7). Wagę β wyznacza formuła Silvera z hiperparametrem b^2 ; klasyczna wartość $b^2 = 10^{-4}$ okazała się nieoptymalna (zbyt silna dominacja Q-RAVE), więc po zamiataniu $b^2 \in \{10^{-6}, \dots, 10^{-1}\}$ wybraliśmy $b^2 = 10^{-2}$. Eksponowany jako `RaveAgent(iterations, c, rave_k, seed)`, gdzie `rave_k` jest interpretowane jako b^2 .

3.4 UCT + Progressive Bias

Formułę selekcji z malejącym członem biasu opisaliśmy w sekcji 2. Wartość heurystyczną ruchu wyznaczamy z tej samej funkcji porządkowania ruchów $\pi(m)$, której używa gracz alpha-beta, znormalizowanej do $[0, 1]$: $H(m) = \text{clip}_{[0,1]}(\pi(m)/1000)$. Dzięki wspólnemu π ta sama wiedza domenowa zasila zarówno Progressive Bias, jak i porządkowanie ruchów w alpha-beta. Wagę biasu ustaliliśmy na $w_{\text{bias}} = 1,0$. Eksponowany jako `ProgressiveBiasAgent(iterations, c, bias_weight, seed)`.

3.5 Heurystyczny gracz alpha-beta

Funkcja ewaluacji. $f : \text{State} \times \text{Player} \rightarrow \mathbb{R}$, dodatnia gdy korzystna dla *Player*. Ocena dla każdej strony sumuje wkład pojedynczych pionków oraz czynniki globalne; dla strony białej $v_W = \sum_p a_p \cdot (1 + 0,3 d_p) \cdot k_p \cdot f_p \cdot g_p$, gdzie dla pionka p w rzędzie r , kolumnie c :

- a_p – waga zaawansowania, *wykładnicza* względem rzędu: $[1, 1, 2, 4, 8, 16, 32, 64]$ (pionek bliżej linii awansu jest wielokrotnie cenniejszy),
- $d_p \in \{0, 1, 2\}$ – liczba własnych obrońców na przekątnych za pionkiem,
- k_p – bonus centralności (1,15 dla kolumn $\{3, 4\}$, 1,05 dla $\{2, 5\}$, inaczej 1,0),
- f_p – 1,5 przy swobodnej ścieżce awansu (brak pionka przeciwnika w rozszerzającym się trójkącie przed pionkiem), inaczej 1,0,
- g_p – 0,8 przy izolacji (brak własnych sąsiadów), inaczej 1,0.

Dodatkowo groźba natychmiastowego przebicia (pionek w przedostatnim rzędzie z legalnym ruchem na rząd awansu) dodaje +200. Czynniki globalne: $f = (v_W - v_B) + 0,3 \cdot \Delta_{\text{mobilność}} + 2,0 \cdot \Delta_{\text{materiał}}$, gdzie $\Delta_{\text{mobilność}}$ to różnica liczby legalnych ruchów obu stron, a $\Delta_{\text{materiał}}$ – różnica liczby pionków. Dla stanu terminalnego zwracamy $\pm\infty$.

3.6 Architektura implementacji

Stos technologiczny. Rdzeń obliczeniowy (silnik gry, wszystkie agenci, ewaluacja) zaimplementowany w języku Rust. Orchestracja eksperymentów, GUI i analiza wyników w Pythonie 3.12. Wiązanie Rust $\leftrightarrow$ Python przez PyO3 0.22 z budowaniem przez maturin.

Struktura repozytorium.

```
breakthrough-mcts/
  src/                                # Rust
    game.rs                          # Bitboard GameState
    mcts/{uct,rave,progressive_bias}.rs
    heuristic.rs                      # alpha-beta
  python/breakthrough/
    reference_game.py                # Walidacja krzyżowa
    gui.py                          # Pygame GUI
    experiments/                    # run_h{1..6}.py
  tests/test_rust_vs_python.py       # Walidacja silnika
  notebooks/                        # Skrypty analityczne
  report/                           # Niniejszy dokument
```

Reprodukowalność. Generator: Xoshiro256++ [9]. Master seed wyprowadza deterministycznie seedy partii (SHA-256). Każda partia ma jawnie zapisany seed w logu JSONL.

Decyzja o zrównolegleniu. MCTS jest sekwencyjny, zrównoleglenie odbywa się wyłącznie na poziomie batchowania eksperymentów (ProcessPoolExecutor).

3.7 Interfejs graficzny (człowiek vs AI)

Do jakościowej oceny agentów i zbierania partii z ludźmi zaimplementowaliśmy graficzny interfejs w bibliotece Pygame (python -m breakthrough.gui).

Ekran startowy (rys. 1) pozwala wybrać algorytm przeciwnika (UCT, RAVE, Progressive Bias, heurystyka, gracz losowy), kolor gracza (białe, czarne lub losowo) oraz parametry agenta: budżet iteracji z presetów (10k–1M), stałą eksploracji c , a dla poszczególnych wariantów dodatkowo b^2 , wagę biasu lub głębokość przeszukiwania.

Podczas rozgrywki (rys. 2) plansza 8×8 jest renderowana z perspektywy gracza; kliknięcie pionka podświetla jego legalne ruchy (kropka – ruch, pierścień – bicie), a ostatni ruch jest zaznaczony. Panel boczny prezentuje konfigurację przeciwnika oraz statystyki partii na żywo: numer ruchu, czas gry i łączny czas namysłu obu stron. Ruch AI liczony jest w osobnym procesie (ProcessPoolExecutor), dzięki czemu interfejs pozostaje responsywny nawet przy dużych budżetach iteracji.

Po zakończonej partii gracz wystawia ocenę trudności (skala 1–10), a całość – log ruchów z czasami namysłu oraz wiersz podsumowania (przeciwnik, wynik, kolor, ocena trudności) – zapisywana jest do `results/human_games/`. Analiza zebranych partii znajduje się w sekcji 5.8.

Breakthrough — Game Setup

Player name

Click to enter your name...

Algorithm

UCT RAVE Prog. Bias Heuristic Random

Play as

White Black Random

Parameters

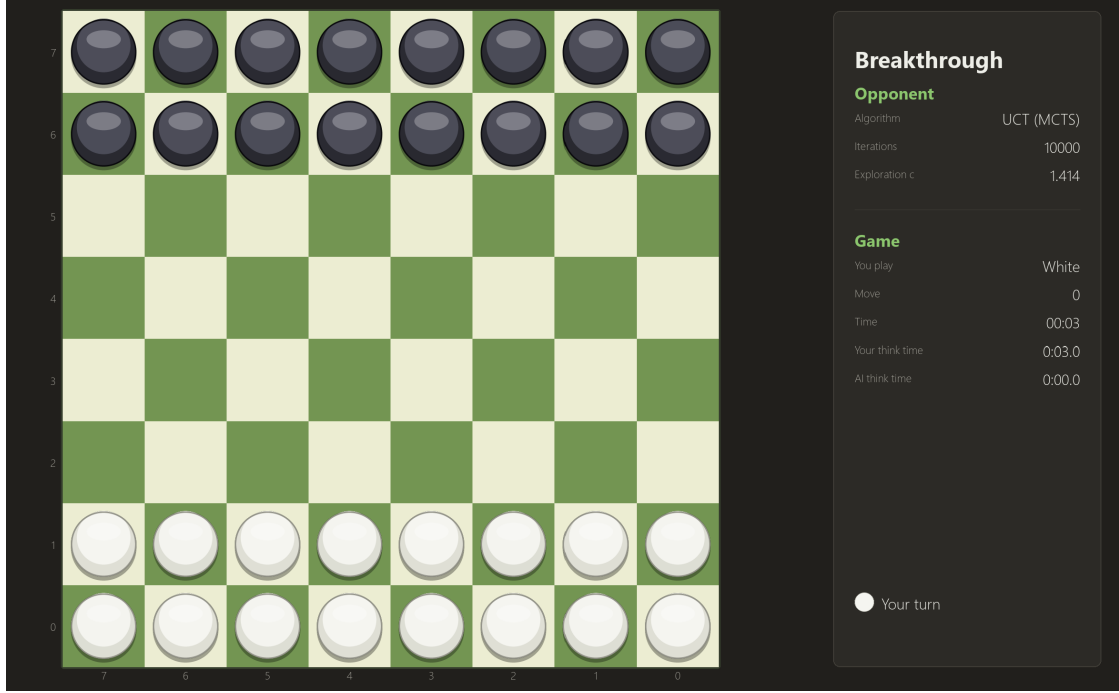
Iterations 10k 50k 100k 200k 500k 800k 1M

Exploration c - 1.41 +

Start Game

Click a value to type it (Enter to confirm) · ESC to quit

Rysunek 1: Ekran konfiguracji partii: wybór algorytmu przeciwnika, koloru gracza i parametrów.



Rysunek 2: Widok rozgrywki: plansza 8×8 oraz panel boczny z konfiguracją przeciwnika i statystykami partii na żywo.

4 Plan eksperymentów

W tej sekcji przedstawiamy konfigurację każdego z sześciu eksperymentów odpowiadających hipotezom H1–H6.

4.1 Wspólna konfiguracja

Wszystkie eksperymenty mają tę samą strukturę:

- **Format turnieju:** każdy z każdym między k agentami $A_1, \dots, A_k$, gdzie każda uporządkowana para (A_i, A_j) z $i \neq j$ gra $n_{\text{games}}/2$ partii z A_i jako białymi i $n_{\text{games}}/2$ partii z A_i jako czarnymi.
- **Master seed** (osobny dla każdego eksperymentu) $\rightarrow$ deterministycznie wyprowadzony seed na partię.
- **Logowanie:** format JSONL, zapisywany append-only po każdej partii. Każda linia zawiera konfigurację obu agentów, seed, wynik, liczbę ruchów, czas decyzji każdego z graczy.
- **Wznawialność:** przy ponownym uruchomieniu skryptu, partie z identyfikatorem już obecnym w pliku JSONL są pomijane.
- **Zrównoleglenie:** `ProcessPoolExecutor` z domyślną liczbą procesów $= n_{\text{cores}} - 1$. *MCTS* wewnątrz pojedynczej partii pozostaje sekwencyjny.

4.2 H1: RAVE vs bazowy UCT

Konfiguracja. 8 agentów: $\{\text{UCT}, \text{RAVE}\} \times \{5000, 10\,000, 50\,000, 100\,000\}$ iteracji. Stała eksploracji $c = \sqrt{2}$. Hiperparametr RAVE: $b^2 = 0,01$ (dobrany empirycznie – patrz sekcja 7).

Liczba partii. 100 partii na parę agentów (50 na każdą stronę) $\Rightarrow \binom{8}{2} = 28$ par; łącznie 2800 partii. Master seed: 42.

Oczekiwania. Zgodnie z hipotezą H1: RAVE wygrywa z UCT przy każdym z testowanych budżetów.

4.3 H2: Progressive Bias vs bazowy UCT

Konfiguracja. 8 agentów: $\{\text{UCT}, \text{Progressive Bias}\} \times \{5000, 10\,000, 50\,000, 100\,000\}$ iteracji. Stała eksploracji $c = \sqrt{2}$. Waga bias termu: $w_{\text{bias}} = 1,0$.

Liczba partii. 100 partii na parę agentów $\Rightarrow \binom{8}{2} = 28$ par; łącznie 2800 partii. Master seed: 43.

Oczekiwania. Progressive Bias wygrywa z UCT przy każdym budżecie, z możliwie większą przewagą przy niskich budżetach.

4.4 H3: Heurystyka vs bazowy UCT

Konfiguracja. Agenci: heurystyka alpha-beta z głębokością 5 oraz UCT z budżetami $\{500, 1000, 5000, 10\,000, 50\,000, 100\,000\}$. 100 partii na parę $\Rightarrow \binom{7}{2} = 21$ par; łącznie 2100 partii. Master seed: 44.

Oczekiwania. Pierwsza część hipotezy: heurystyka wygrywa z UCT przy niskich budżetach (≤ 5000 iteracji). Druga część: heurystyka przegrywa z UCT przy wysokich budżetach ($\geq 50\,000$ iteracji).

4.5 H4: Przewaga pierwszego gracza

Konfiguracja. Symetryczne mecze UCT(5000) vs UCT(5000) na planszy 8×8 – ten sam agent (identyczny budżet i hiperparametry) gra jako biały i jako czarny, przy deterministycznie różnych seedach. Master seed: 45. Łącznie 500 partii.

Oczekiwania. Białe (pierwszy gracz) mają niewielką, lecz istotną statystycznie przewagę ($> 50\%$ wygranych), analogicznie do obserwacji z szachów.

4.6 H5: Strojenie stałej eksploracji c

Konfiguracja. 5 agentów UCT z budżetem 10 000 iteracji, różniących się jedynie stałą eksploracji $c \in \{0,5, 1,0, 1,5, 2,0, 3,0\}$. 100 partii na parę $\Rightarrow \binom{5}{2} = 10$ par; łącznie 1000 partii. Master seed: 46.

Oczekiwania. Istnieje optymalna wartość c^* ; win-rate jako funkcja c ma kształt odwróconego U (spadek przy zbyt małym i zbyt dużym c). Wartość teoretyczna $c = \sqrt{2} \approx 1,41$ jest naturalnym kandydatem na okolicę optimum.

4.7 H6: Czas decyzji w funkcji numeru ruchu

Konfiguracja. 3 agenci UCT z budżetami $\{5000, 10\,000, 50\,000\}$ iteracji (różne budżety, by zbadać skalowanie czasu). Dla każdej partii rejestrowany jest czas decyzji osobno dla każdego ruchu. 100 partii na parę $\Rightarrow \binom{3}{2} = 3$ pary; łącznie 300 partii. Master seed: 47.

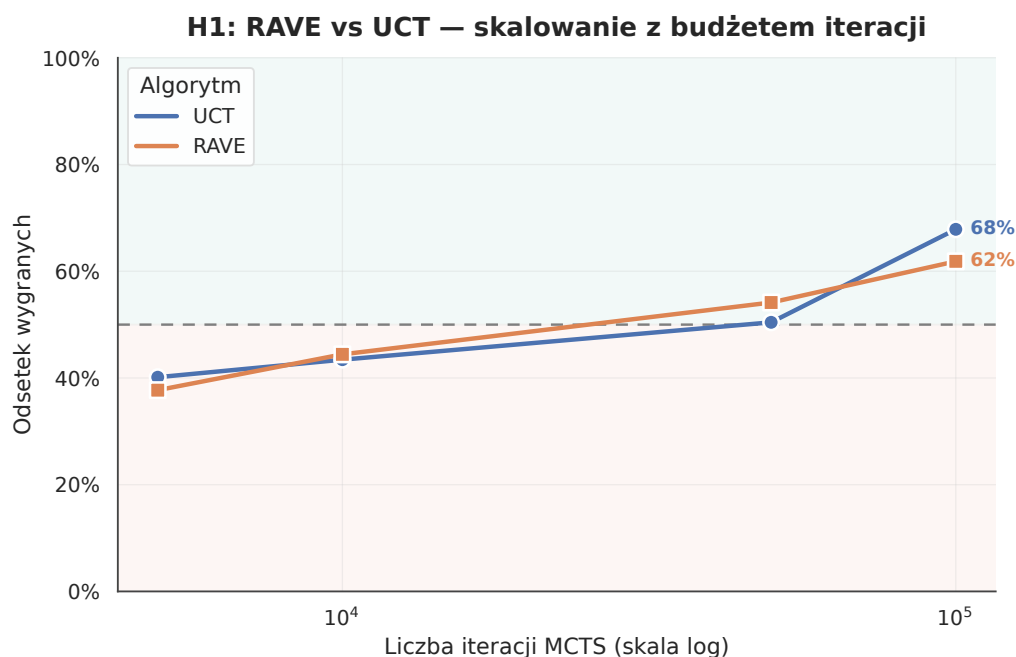
Oczekiwania. Przy stałym budżecie iteracji średni czas na ruch rośnie w środkowej fazie gry (ruchy 15–40), gdy branching factor jest największy, i maleje w otwarciu oraz końcówce.

Reprodukowalność. Wszystkie eksperymenty są w pełni reprodukowalne. Aby powtórzyć dany eksperyment z dokładnie tymi samymi wynikami:

```
python -m breakthrough.experiments.run_h1 # H1
python -m breakthrough.experiments.run_h2 # H2
python -m breakthrough.experiments.run_h3 # H3
python -m breakthrough.experiments.run_h4 # H4
python -m breakthrough.experiments.run_h5 # H5
python -m breakthrough.experiments.run_h6 # H6
```

5 Wyniki

5.1 H1: RAVE vs bazowy UCT



Rysunek 3: Skalowanie odsetków wygranych RAVE i UCT z budżetem iteracji. Punkty pokazują ogólny win-rate każdego agenta we wszystkich rozegranych przez niego partiach. Linia 50% oznacza neutralność. Obie metody zyskują z budżetem; RAVE nie wyprzedza UCT przy żadnym budżecie.

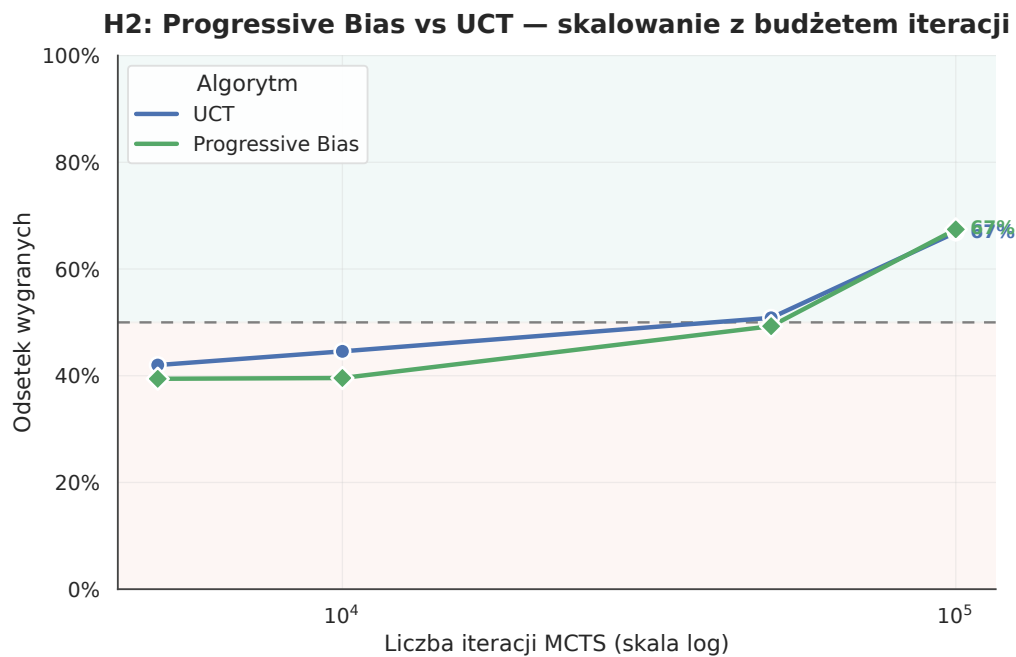
Dane liczbowe – bezpośrednie mecze RAVE vs UCT przy tym samym budżecie (n=100 partii na budżet):

Budżet	RAVE wygrywa	Win-rate	p (vs 0,5)	Status
5 000	49/100	49,0%	0,92	bez różnicy
10 000	48/100	48,0%	0,76	bez różnicy
50 000	54/100	54,0%	0,48	bez różnicy
100 000	39/100	39,0%	0,035	poniżej 50%

Hipoteza H1 jest *odrzucona*: RAVE nie wygrywa istotnie z bazowym UCT przy żadnym budżecie, a przy 100 000 iteracji *przegrywa* istotnie statystycznie (39,0%, $p = 0,035$). Rezultat jest odmienny od oczekiwań literaturowych z dziedziny Go, gdzie RAVE przewyższa bazowy UCT przy każdym praktycznym budżecie [6].

Interpretujemy to następująco: RAVE akumuluje wartości AMAF przez wszystkie pozycje w drzewie i symulacjach, zakładając przesuwalność wartości ruchu. Założenie to zachodzi w Go (kamień postawiony w punkcie X ma podobną wartość w różnych kontekstach), ale załamuje się w Breakthrough, gdzie wartość ruchu (np. „pionek z pola X na Y ”) silnie zależy od reszty pozycji. Sygnał AMAF jest więc mało informatywny, a przy największym budżecie działa niekorzystnie, podczas gdy bazowy UCT efektywnie wykorzystuje rosnący budżet do precyzowania szacunków per-stan. Poprawność samej implementacji RAVE potwierdziliśmy niezależnie (sekcja 7): przy zneutralizowanym członie AMAF ($\beta \rightarrow 0$) RAVE odtwarza siłę UCT, co wskazuje, że obserwowany spadek pochodzi z heurystyki AMAF, a nie z błędu w szkielecie algorytmu.

5.2 H2: Progressive Bias vs bazowy UCT



Rysunek 4: Skalowanie odsetków wygranych Progressive Bias i UCT (8×8). Człon bias w selekcji oparty na heurystyce porządkowania ruchów ($\pi/1000$). Krzywe PB i UCT pokrywają się na całym zakresie budżetów – brak statystycznie istotnej różnicy.

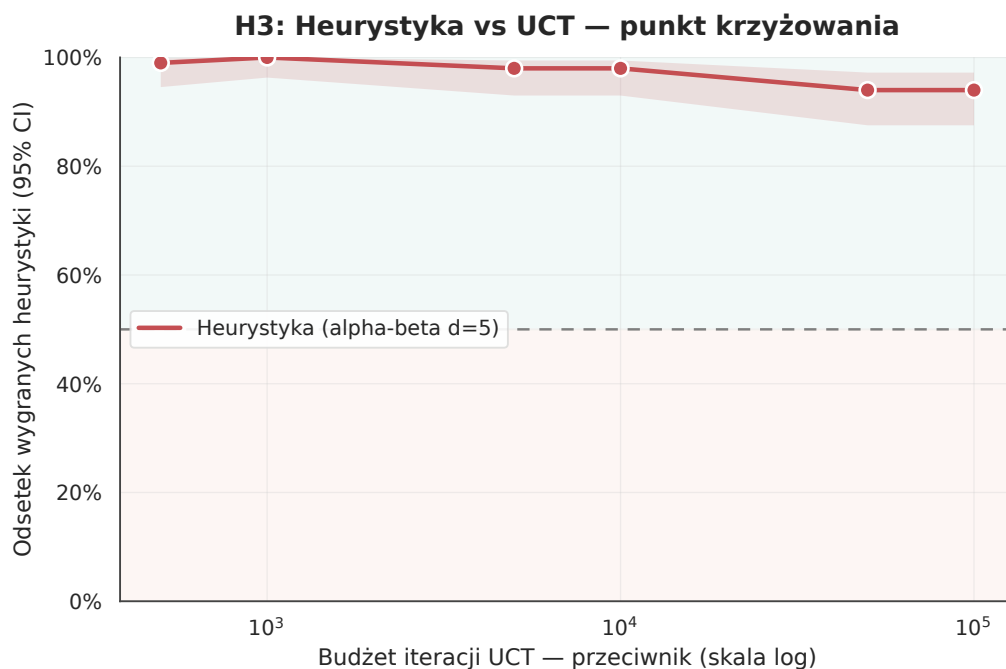
Dane liczbowe – bezpośrednie mecze PB vs UCT przy tym samym budżecie ($n=100$ partii na budżet):

Budżet	PB wygrywa	Win-rate	p (vs 0,5)	Status
5 000	50/100	50,0%	1,00	bez różnicy
10 000	43/100	43,0%	0,19	bez różnicy
50 000	55/100	55,0%	0,37	bez różnicy
100 000	46/100	46,0%	0,48	bez różnicy

W żadnym z testowanych budżetów PB nie pokonuje UCT istotnie statystycznie – wszystkie wyniki oscylują wokół 50% ($p > 0,19$). Hipoteza H2 jest zatem *niejednoznaczna*: nie można odrzucić hipotezy zerowej o braku różnicy względem UCT.

Interpretacja: heurystyka użyta w członie bias to znormalizowana funkcja porządkowania ruchów – prostsza niż pełna funkcja ewaluacji w agentcie heurystycznym (sekcja 5.3). Wiedza domenowa wstrzykiwana przez PB jest więc ograniczona, a jej wpływ dodatkowo zanika wraz z liczbą odwiedzin węzła ($H/(n+1)$).

5.3 H3: Heurystyka alpha-beta vs UCT



Rysunek 5: Odsetek wygranych heurystyki (alpha-beta z głębokością 5) względem rosnącego budżetu iteracji UCT na planszy 8×8. Oś X logarytmiczna. Heurystyka wygrywa z UCT w całym testowanym zakresie; przy 100 000 iteracji nadal wygrywa 94% partii.

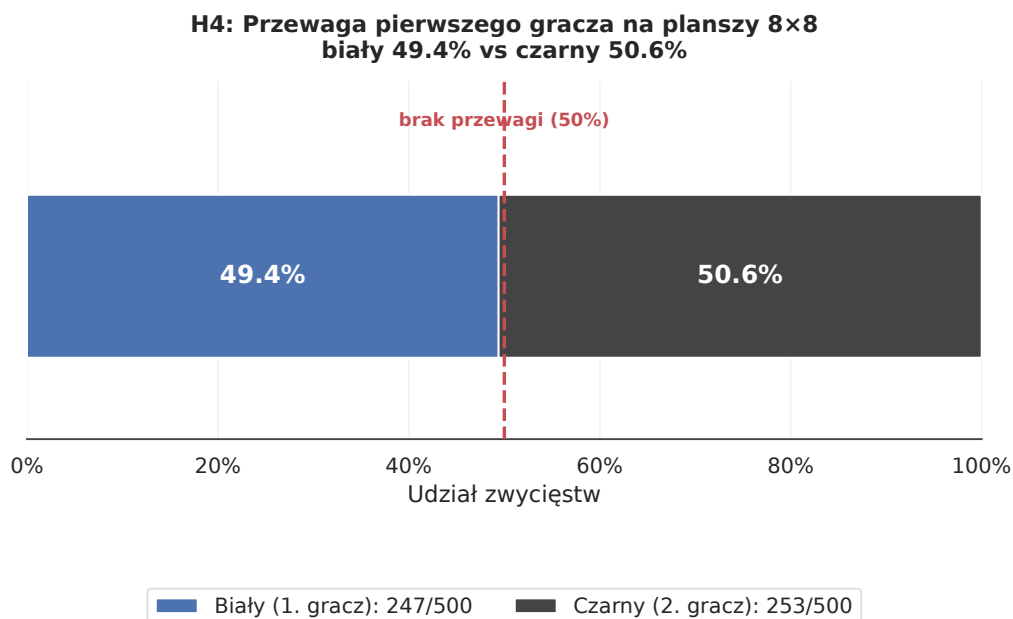
Dane liczbowe (n=100 partii na budżet, bezpośrednie mecze heurystyka vs UCT):

Budżet UCT	Heur wygrywa	Win-rate	95% CI	UCT czas/ruch
500	99/100	99,0%	[94,6; 99,8]	~0,005 s
1 000	100/100	100,0%	[96,3; 100]	~0,01 s
5 000	98/100	98,0%	[93,0; 99,4]	~0,05 s
10 000	98/100	98,0%	[93,0; 99,4]	~0,10 s
50 000	94/100	94,0%	[87,5; 97,2]	~0,50 s
100 000	94/100	94,0%	[87,5; 97,2]	~0,97 s

Pierwsza część hipotezy (wyższość heurystyki przy niskich budżetach) jest potwierdzona. Druga część (przewaga UCT przy wysokich budżetach) nie zostaje osiągnięta: win-rate maleje łagodnie (z 99% do 94%) i pozostaje daleko od 50% nawet przy UCT(100 000), który zajmuje już ~1 s na ruch.

Heurystyka wybiera ruch w ~10 ms, więc cała partia trwa poniżej sekundy – o rzędy wielkości szybciej niż UCT(100 000), a przy tym gra silniej. Pokazuje to przydatność prostych graczy z funkcją ewaluacji w grach taktycznych o umiarkowanym branching factor.

5.4 H4: Przewaga pierwszego gracza



Rysunek 6: Udział zwycięstw białych (pierwszego gracza) i czarnych w symetrycznym turnieju UCT(5000) vs UCT(5000) na planszy 8×8 (n=500). Przerwana linia 50% oznacza brak przewagi.

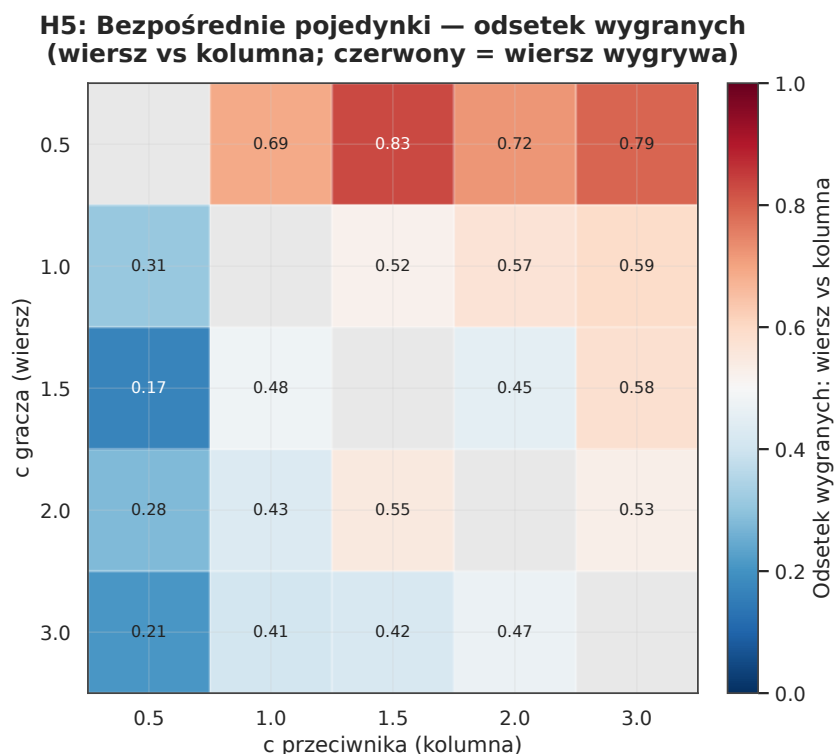
Dane liczbowe (n=500 partii): Białe wygrały $247/500 = 49,4\%$ partii, 95% CI Wilsona [45,0%; 53,8%] (średnia długość partii ok. 59 ruchów). Przedział obejmuje 50%, brak więc istotnej statystycznie przewagi pierwszego gracza.

Hipoteza H4 jest odrzucona: nie stwierdzamy istotnej statystycznie przewagi białych. Szerokość CI (ok. $\pm 4,4$ pp) jest wystarczająco wąska, by uznać, że ewentualna „prawdziwa” przewaga jest mniejsza niż 4 pp – co metodologicznie odpowiada brakowi przewagi praktycznej.

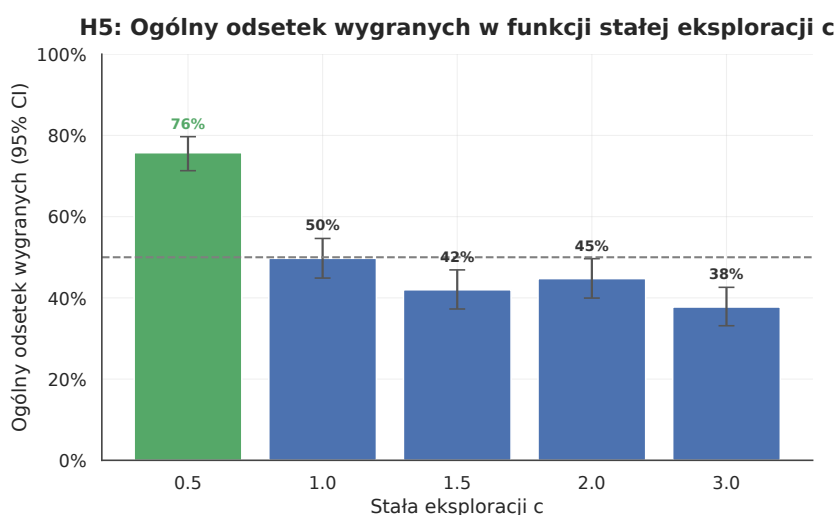
Interpretacja:

- Przy MCTS o zbliżonej sile oboje gracze grają sub-optymalnie w sposób przybliżenie symetryczny – przewaga tempa pierwszego ruchu zostaje „wyzerowana” przez błędy obu stron.
- Możliwe, że przy perfekcyjnej grze (jak w rozwiązanych wariantach 6×5 [1] i 6×6 [2], gdzie wygrywa pierwszy gracz) białe rzeczywiście mają przewagę, lecz UCT z 5000 iteracji nie zbliża się do tej granicy.

5.5 H5: Stała eksploracji c



Rysunek 7: Macierz win-rate dla turnieju każdy z każdym pięciu wartości stałej eksploracji c (UCT 10 000 iteracji, 8×8). Wiersz – c gracza, kolumna – c przeciwnika. Skala dwubiegunowa wycentrowana na 50%: czerwień oznacza przewagę gracza z wiersza, błękit – przeciwnika z kolumny. Wiersz $c = 0,5$ jest wyraźnie czerwony i dominuje całą tabelę.



Rysunek 8: Ogólny win-rate każdej wartości c (uśredniony po obu kolorach i wszystkich przeciwnikach). Trend jest w przybliżeniu monotonicznie malejący; brak wewnętrznego maksimum w badanym zakresie.

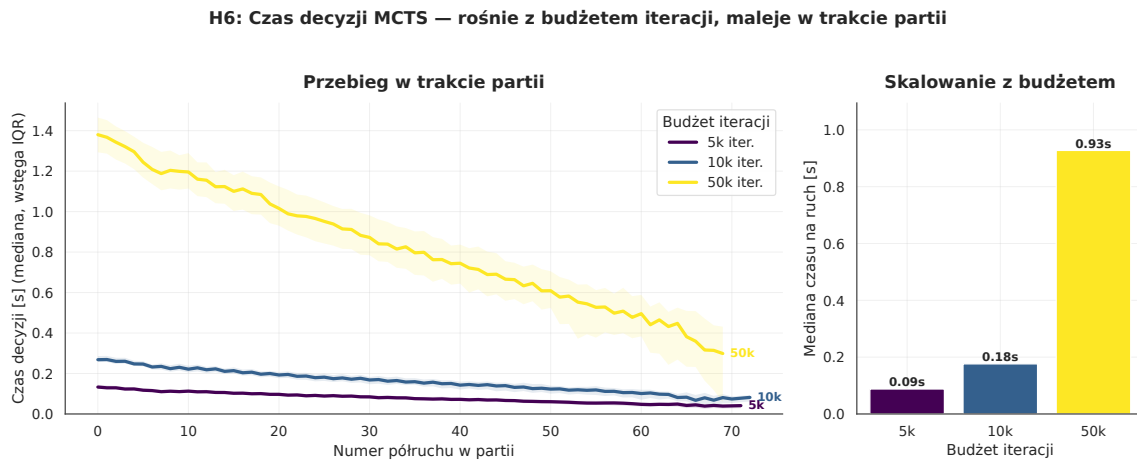
Dane liczbowe ($n=400$ partii na wartość c):

c	Win-rate	95% CI	p (vs 0,5)	Status
0,5	75,7%	[71,3; 79,7]	$< 10^{-4}$	najlepsza
1,0	49,7%	[44,9; 54,6]	0,96	neutralna
1,5	42,0%	[37,3; 46,9]	0,0016	poniżej 50%
2,0	44,8%	[40,0; 49,6]	0,040	poniżej 50%
3,0	37,8%	[33,1; 42,6]	$< 10^{-4}$	najgorsza

Hipoteza H5 (istnienie wewnętrznego optimum c^* o kształcie odwróconego U) *nie znajduje potwierdzenia* w badanym zakresie. Najsilniejsza okazuje się *najniższa* testowana wartość $c = 0,5$ (75,7% wygranych, wysoce istotnie), a win-rate maleje wraz ze wzrostem c – nie obserwujemy spadku przy małym c , który zamykałby krzywą w kształt odwróconego U. Co więcej, teoretyczna wartość $c = \sqrt{2} \approx 1,41$ (okolice $c = 1,5$) wypada wyraźnie poniżej 50%.

Interpretacja: Breakthrough jest grą silnie taktyczną, w której kluczowe są konkretne, wymuszone warianty, a nie szerokie zwiedzanie drzewa. Niski c (nacisk na eksploatację) pozwala MCTS głębiej dążyć obiecujące linie, co przekłada się na realną przewagę. Optymalne c^* leży zatem *na lub poniżej* dolnej granicy testowanego zakresu – co stanowi praktyczną wskazówkę do strojenia UCT w tej grze i sugeruje rozszerzenie eksperymentu o $c \in \{0,1, 0,25\}$.

5.6 H6: Czas decyzji w funkcji numeru ruchu



Rysunek 9: Czas decyzji UCT w funkcji numeru póhruchu (8×8), w rozbiściu na budżet iteracji. Każdy póhruch przypisano do budżetu agenta, który go wykonał. Lewy panel: mediana czasu wraz z pasmem międzykwartylowym w trakcie partii – czas jest najwyższy w otwarciu i maleje monotonicznie. Prawy panel: skalowanie mediany czasu na ruch z budżetem iteracji.

Mediana czasu na ruch wg budżetu iteracji:

Budżet UCT	Mediana czasu/ruch
5 000	0,09 s
10 000	0,18 s
50 000	0,93 s

Hipoteza H6 (czas decyzji rośnie w środkowej fazie gry) jest *odrzucona*. Wbrew oczekiwaniom, czas na ruch jest *najwyższy w otwarciu* i maleje monotonicznie w miarę postępu partii – nie obserwujemy maksimum w ruchach 15–40. Wyjaśnienie tkwi w naturze kosztu pojedynczej iteracji MCTS: przy stałym budżecie iteracji dominującym kosztem jest długość losowej

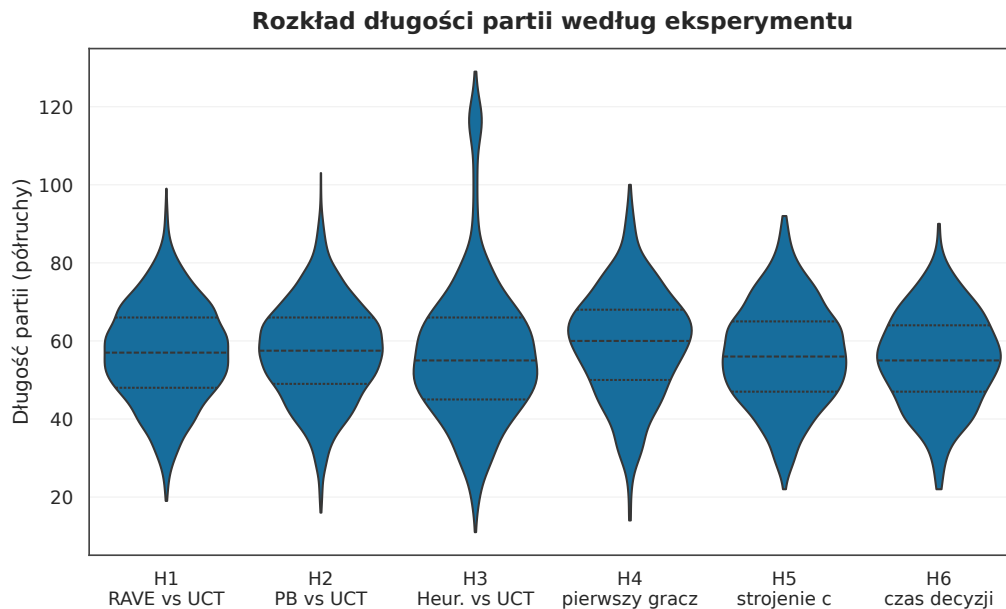
symulacji (rolloutu do stanu terminalnego), a ta jest najdłuższa na początku gry i kurczy się wraz ze zbliżaniem do końcówki. Większy branching factor w środkowej fazie *nie* podnosi kosztu ruchu, ponieważ liczba iteracji jest ustalona. Zgodnie z oczekiwaniem skaluje się natomiast czas z budżetem (mediana 0,09 s $\rightarrow$ 0,93 s dla 5k $\rightarrow$ 50k iteracji).

Uwaga metodologiczna. Ponieważ w tym eksperymencie kojarzymy agentów o różnych budżetach, każdy półruch przypisujemy do budżetu agenta, który go faktycznie wykonał (ruchy parzyste – biały, nieparzyste – czarny). Dzięki temu krzywe odzwierciedlają rzeczywisty koszt decyzji danego budżetu, a nie artefakt naprzemiennej gry graczy o różnej sile.

5.7 Analizy przekrojowe

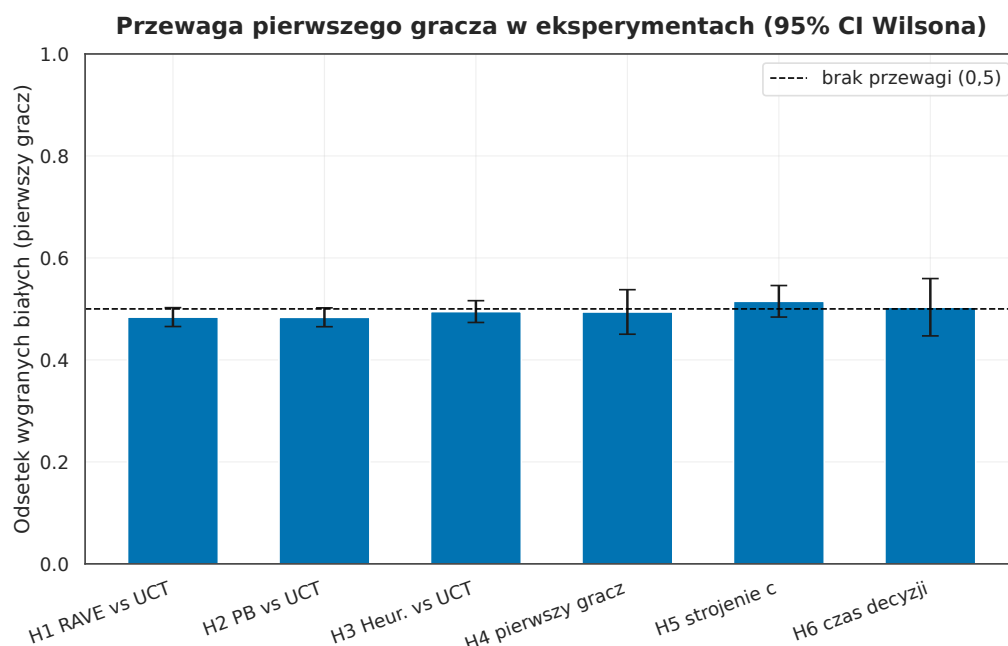
Poza testami pojedynczych hipotez zebraliśmy kilka obserwacji przekrojowych po wszystkich sześciu zbiorach danych (H1–H6).

Długość partii. Rozkłady długości partii są zbliżone we wszystkich eksperymentach (rys. 10): mediana 55–60 półruchów. Partie na planszy 8×8 rzadko kończą się przed 40. półruchem i niemal nigdy nie przekraczają 75 – przewaga taktyczna jest realizowana stopniowo przez marsz pionków.



Rysunek 10: Rozkłady długości partii (skrzypce) w sześciu eksperymentach.

Przewaga pierwszego gracza – ujęcie zbiorcze. Odsetek wygranych białych mieści się w wąskim paśmie 48–52% we wszystkich sześciu eksperymentach, a każdy 95% CI Wilsona obejmuje 50% (rys. 11). Potwierdza to wniosek z H4 w szerszym kontekście (różne algorytmy, budżety i stałe c): przy MCTS o zbliżonej sile przewaga pierwszego ruchu jest pomijalna.



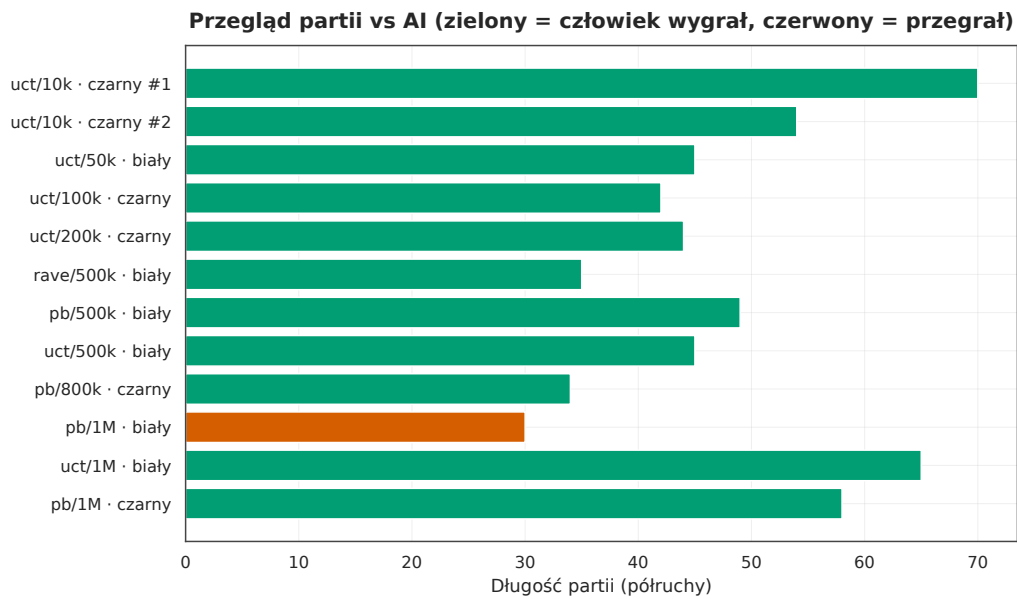
Rysunek 11: Odsetek wygranych białych w każdym z eksperymentów (95% CI Wilsona). Wszystkie przedziały obejmują 50%.

5.8 Partie człowiek vs AI

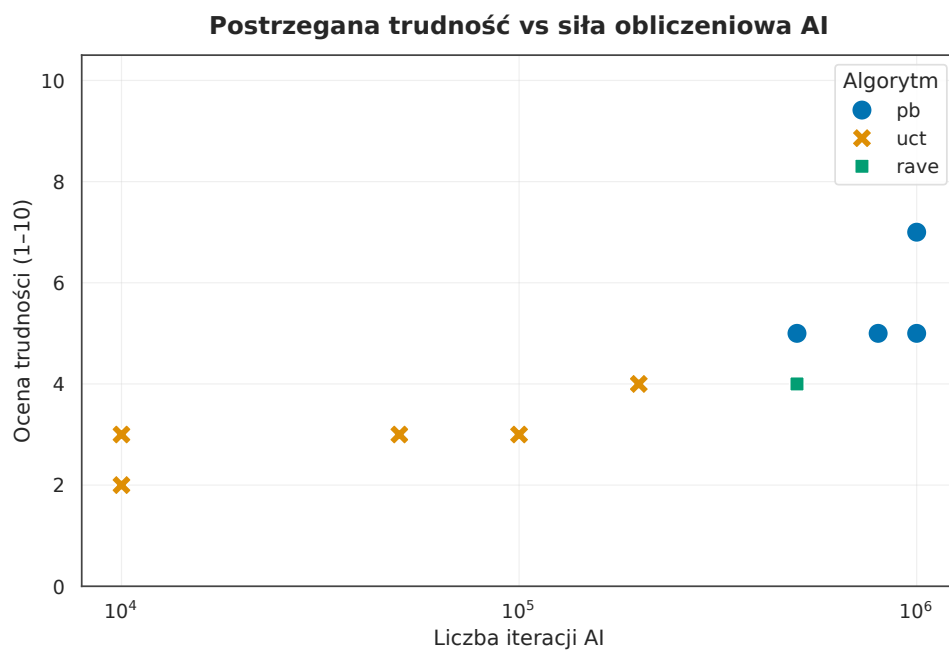
Dla jakościowego zilustrowania siły agentów udostępniliśmy grę przez interfejs graficzny (sekcja 3.7) i zebraliśmy 12 partii człowiek–AI. Jest to demonstracja narzędzia, a nie rygorystyczny eksperyment: próba jest mała, a poziom ludzkiego gracza niejednorodny, więc wyników nie należy traktować jako pomiaru siły w sensie statystycznym.

Człowiek wygrał 11 z 12 partii przeciwko agentom MCTS o budżetach od 10 000 do 1 000 000 iteracji (rys. 12). Wynik jest spójny z H3: czysty UCT, pozbawiony wiedzy domenowej, gra Breakthrough taktycznie słabo nawet przy bardzo dużych budżetach. Postrzegana trudność (ocena 1–10 wystawiana po partii) miała medianę 4,5 i nie rosła wyraźnie z budżetem iteracji (rys. 13).

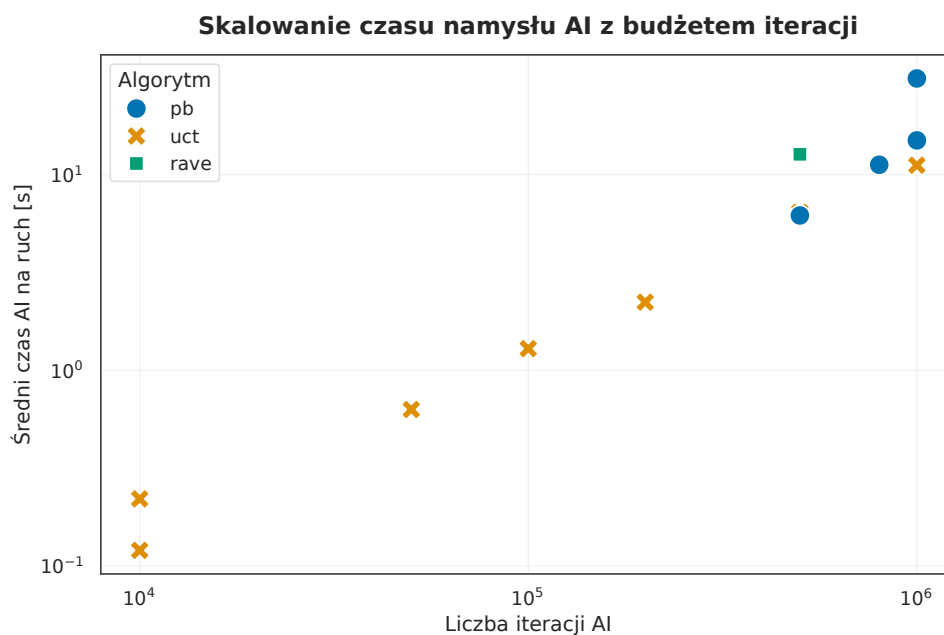
Czas namysłu AI skaluje się z budżetem – od ułamków sekundy przy 10k iteracji do kilkadziesiąt sekund na ruch przy 1M (rys. 14). Człowiek namyślał się medianowo $\sim 5,2$ s na ruch (AI $\sim 4,2$ s), przy czym ludzki czas jest największy w fazie środkowej, gdy pozycja jest najbardziej taktyczna (rys. 15).



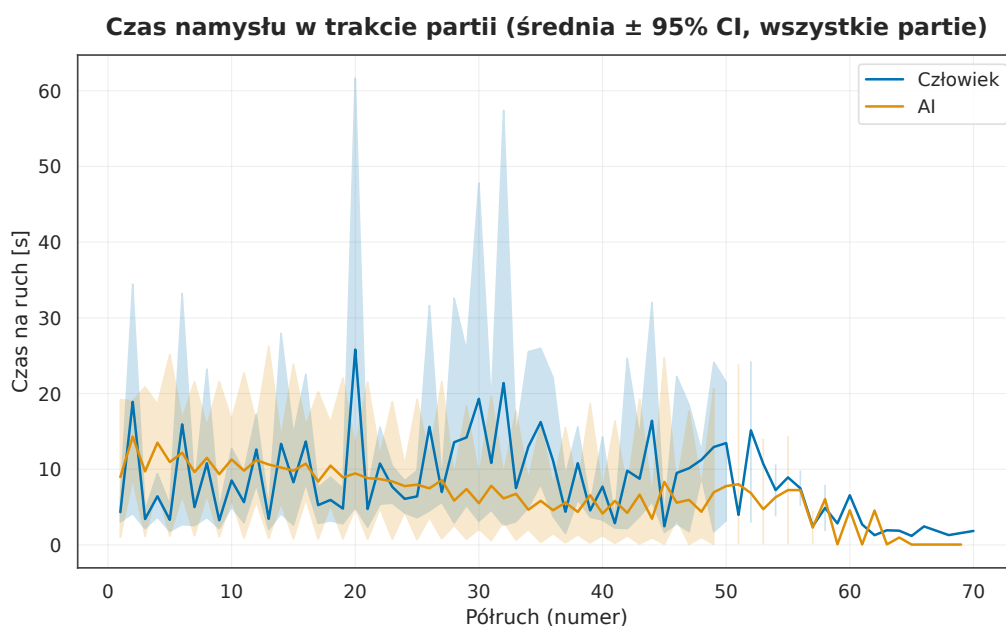
Rysunek 12: Przegląd 12 partii człowiek–AI (zielony – człowiek wygrał, czerwony – przegrał); oś pozioma to długość partii.



Rysunek 13: Postrzegana trudność (1–10) względem budżetu iteracji AI.



Rysunek 14: Średni czas AI na ruch względem budżetu iteracji (osie logarytmiczne).



Rysunek 15: Czas namysłu na ruch w trakcie partii: człowiek vs AI (średnia $\pm$ 95% CI po wszystkich partiach).

6 Wnioski

6.1 Synteza i kluczowe odkrycie

Najbardziej zaskakującym wynikiem projektu jest **nieskuteczność RAVE w Breakthrough 8×8**. Pokazuje to, że standardowe techniki rozwoju MCTS są domain-specific: założenie „dobrego ruchu zagranego gdziekolwiek” (kluczowe dla AMAF) działa świetnie w Go, lecz załamuje się w grze, gdzie te same fizyczne ruchy mają radykalnie różną wartość w zależności od pozycji. Co istotne, wynik ten nie jest artefaktem implementacji – poprawność szkieletu RAVE

potwierdziliśmy testem $\beta \rightarrow 0$ (odtworzenie siły UCT), więc spadek pochodzi z samej heurystyki AMAF.

Drugim ważnym wnioskiem jest **przygniatająca przewaga prostej heurystyki alpha-beta** dla tej klasy gier. Heurystyka oparta na bogatej funkcji ewaluacji (wykładnicze zaawansowanie, groźba przebicia, obrońcy, centralność, swobodna ścieżka, izolacja, mobilność i materiał – Algorytm 1) z głębokością 5 i porządkowaniem ruchów dominuje MCTS w całym praktycznym zakresie iteracji (94% nawet przy 100 000 iteracji UCT), grając przy tym rzędy wielkości szybciej.

Trzecim, praktycznym odkryciem jest **korzyść z niskiej stałej eksploracji** (H5): w taktycznym Breakthrough $c = 0,5$ wyraźnie przewyższa kanoniczne $c = \sqrt{2}$, co sugeruje, że gra premiuje eksploatację (głębsze drażnienie wymuszonych wariantów) nad eksploracją.

6.2 Propozycje dalszych prac

- **Polityka rolloutowa oparta na heurystyce.** Jednym z najprostszych rozszerzeń byłaby zamiana czysto losowych symulacji na epsilon-greedy po heurystyce (np. ulubione ruchy z funkcji ewaluacji). Powinno to drastycznie poprawić wszystkich agentów MCTS i może zmienić rezultat H1 (RAVE z heurystycznymi rolloutami).
- **MAST i NAST.** Alternatywne techniki uczące się polityki symulacyjnej – Move-Average Sampling Technique [10] – mogłyby przewyższyć heurystykę w niskim/średnim zakresie iteracji. To naturalna kontynuacja H2.
- **Krzyżowanie z deep RL.** Implementacja prostej sieci ewaluacyjnej (jak w AlphaZero, lecz z mniejszą architekturą) i jej porównanie z heurystyką alpha-beta.

7 Wykorzystanie modeli językowych LLM

Modele językowe wspomagały nas w trzech obszarach:

- redakcji niniejszego raportu,
- implementacji graficznego interfejsu użytkownika (GUI),
- pisaniu kodu generującego część wykresów.

Bibliografia

- [1] Abdallah Saffidine, Nicolas Jouandeau i Tristan Cazenave. “Solving Breakthrough with Race Patterns and Job-Level Proof Number Search”. W: *Advances in Computer Games (ACG 2011)*. T. 7168. LNCS. Springer, 2011, s. 196–207.
- [2] Yngvi Björnsson i in. “Solving Breakthrough for the 6×6 Board”. W: *ICGA Journal* (2026). Online first, March 2026. DOI: [10.1177/13896911261430592](https://doi.org/10.1177/13896911261430592).
- [3] Rémi Coulom. “Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search”. W: *Computers and Games (CG 2006)*. T. 4630. LNCS. Springer, 2006, s. 72–83.
- [4] Levente Kocsis i Csaba Szepesvári. “Bandit Based Monte-Carlo Planning”. W: *European Conference on Machine Learning (ECML 2006)*. T. 4212. LNCS. Springer, 2006, s. 282–293.
- [5] Cameron B. Browne i in. “A Survey of Monte Carlo Tree Search Methods”. W: *IEEE Transactions on Computational Intelligence and AI in Games* 4.1 (2012), s. 1–43.
- [6] Sylvain Gelly i David Silver. “Monte-Carlo Tree Search and Rapid Action Value Estimation in Computer Go”. W: *Artificial Intelligence* 175.11 (2011), s. 1856–1875.
- [7] Guillaume M. J.-B. Chaslot i in. “Progressive Strategies for Monte-Carlo Tree Search”. W: *New Mathematics and Natural Computation* 4.3 (2008), s. 343–357.

- [8] Richard J. Lorentz. “An MCTS Program to Play EinStein Würfelt Nicht!” W: *Advances in Computer Games (ACG 2011)*. T. 7168. LNCS. Springer, 2011, s. 52–59.
- [9] Sebastiano Vigna. “Further Scramblings of Marsaglia’s Xorshift Generators”. W: *Journal of Computational and Applied Mathematics* 315 (2017), s. 175–181.
- [10] Hilmar Finnsson i Yngvi Björnsson. “Simulation-Based Approach to General Game Playing”. W: *AAAI*. 2008, s. 259–264.